UNIDAD DE PROGRAMACIÓN 2

FUNDAMENTOS DE JAVA

Guía didáctica de la UP2 · De PSeInt a Java

```java
import java.util.Scanner;

public class Hola {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        String nombre = teclado.nextLine();
        System.out.println("Hola, " + nombre); // ¡Bienvenido a Java!
    }
}
```

Rubén Pérez Ibáñez

IES Álvaro Falomir · Curso 2026/27

Módulo: Programación (0485) · 1.º DAW

Índice de contenidos

Cómo utilizar esta guía 3

1. De PSeInt a Java 3

2. IntelliJ IDEA: tu primer proyecto 5

3. Estructura de un programa Java 6

4. Comentarios, identificadores y convenciones 8

5. Variables y tipos de datos 9

6. Constantes y literales 11

7. Operadores y expresiones 12

8. Conversiones de tipo 14

9. Salida por pantalla 18

10. Entrada por teclado 20

11. La clase Math 22

12. Errores frecuentes y cómo leerlos 23

13. Ejemplos resueltos paso a paso 24

14. Banco de ejercicios 28

15. Referencia rápida 29

Cómo utilizar esta guía

Esta guía acompaña a la UP2 y tiene un objetivo muy concreto: que pases de escribir algoritmos en PSeInt a escribir **programas reales en Java** que compilan, se ejecutan y resuelven problemas sencillos de entrada, cálculo y salida.

Cada apartado sigue la misma secuencia que ya conoces de la UP1: **problema → idea → código → prueba → ejercicio**. No intentes memorizar toda la sintaxis de golpe: escribe los ejemplos en IntelliJ, ejecútalos, cambia una línea y observa qué ocurre.

> **Objetivo de la unidad**
> Al terminar deberías ser capaz de crear un proyecto en IntelliJ, declarar variables del tipo adecuado, leer datos por teclado, hacer cálculos con operadores, aplicar conversiones de tipo (implícitas y explícitas), mostrar resultados con un formato claro y usar métodos estáticos de clases predefinidas como `Math`.

Si quieres…	Ve al apartado
Ver la equivalencia entre PSeInt y Java	1
Crear y ejecutar tu primer proyecto	2 y 3
Saber qué tipo de dato usar	5
Entender por qué `7 / 2` da 3	7 y 8
Entender el *casting* (int, double, String)	8
Leer datos sin errores raros	10
Consultar la sintaxis rápidamente	15

---

1. De PSeInt a Java

En la UP1 aprendiste a pensar algoritmos. Java no cambia la lógica: cambia la forma de escribirla. La mayoría de lo que hacías en PSeInt tiene una traducción directa.

PSeInt	Java	Comentario
`Algoritmo Saludo … FinAlgoritmo`	`public class Saludo { public static void main(String[] args) { … } }`	El programa vive dentro de una clase y empieza en `main`.
`Definir edad Como Entero`	`int edad;`	En Java es obligatorio declarar el tipo.
`Definir precio Como Real`	`double precio;`	
`Definir nombre Como Cadena`	`String nombre;`	`String` empieza por mayúscula.
`Definir ok Como Logico`	`boolean ok;`	Valores `true` / `false`.
`edad <- 18`	`edad = 18;`	El `=` de Java **asigna**; no compara.
`Escribir "Hola"`	`System.out.println("Hola");`	
`Leer edad`	`edad = teclado.nextInt();`	Necesita un `Scanner`.
`a MOD b`	`a % b`	Resto de la división entera.
`a ^ 2`	`Math.pow(a, 2)` o `a * a`	`^` en Java **no** es potencia.
`Y`, `O`, `NO`	`&&`, `||`, `!`	

> **Tres diferencias que más errores provocan**
> Cada instrucción termina en `;`. Java distingue mayúsculas y minúsculas, así que `Edad` y `edad` son variables distintas. Y los bloques se delimitan con llaves `{ }`, no con palabras como `FinSi`.

1.1. Compilar y ejecutar

PSeInt interpretaba tu algoritmo directamente. Java hace dos pasos:

1. **Compilación.** El compilador (`javac`) revisa el código fuente (`Saludo.java`) y, si no hay errores, genera *bytecode* (`Saludo.class`).
2. **Ejecución.** La máquina virtual de Java (JVM) ejecuta ese *bytecode*. Por eso el mismo programa funciona en Windows, macOS o Linux.

IntelliJ hace ambos pasos cuando pulsas el botón ▶ *Run*. Si hay un error de compilación, el programa **no llega a ejecutarse**: el IDE subraya en rojo la línea y muestra un mensaje en la parte inferior.

1.2. Un algoritmo traducido paso a paso

Este es el algoritmo de la suma de la UP1 y su versión en Java. Fíjate en que la estructura (declarar, leer, calcular, mostrar) es idéntica:

```
Algoritmo Suma
    Definir a, b, resultado Como Entero;
    Escribir "Primer número:";
    Leer a;
    Escribir "Segundo número:";
    Leer b;
    resultado <- a + b;
    Escribir "La suma es ", resultado;
FinAlgoritmo
```

```java
import java.util.Scanner;

public class Suma {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Primer número: ");
        int a = teclado.nextInt();
        System.out.print("Segundo número: ");
        int b = teclado.nextInt();

        int resultado = a + b;
        System.out.println("La suma es " + resultado);
    }
}
```

Si tecleas 3 y 4, el programa muestra `La suma es 7`. Diferencias a notar: el `Scanner` (que en PSeInt no hacía falta crear), el tipo delante de cada variable, el `;` en todas las líneas y que `print` no salta de línea (como `Escribir Sin Saltar`) mientras que `println` sí (como `Escribir`). Aún no hay condicionales ni bucles: llegarán en la UP3, y de momento todo el código va dentro de `main` (los métodos propios llegan en la UP4).

---

2. IntelliJ IDEA: tu primer proyecto

Usaremos **IntelliJ IDEA Community** (gratuito) o la licencia educativa de Ultimate.

2.1. Crear el proyecto

1. *File → New → Project…*
2. Elige **Java**, ponle nombre (`up2-ejercicios`) y una ubicación que recuerdes.
3. *Build system:* **IntelliJ** (más adelante usaremos Maven).
4. *JDK:* elige uno instalado (21 o superior). Si no aparece ninguno, *Download JDK…* y selecciona una distribución (por ejemplo, Eclipse Temurin).
5. Desmarca *Add sample code* si quieres empezar desde cero.

2.2. Crear una clase y ejecutarla

1. Clic derecho sobre la carpeta `src` → *New* → *Java Class* → escribe `Saludo`.
2. Escribe `psvm` y pulsa Tab: IntelliJ genera el método `main`.
3. Dentro de `main`, escribe `sout` y pulsa Tab: genera `System.out.println();`.
4. Ejecuta con el triángulo verde que aparece junto a `main` (o Mayús + F10).

Atajo / plantilla	Qué hace
`psvm` + Tab	Crea `public static void main(String[] args)`
`sout` + Tab	Crea `System.out.println();`
`souf` + Tab	Crea `System.out.printf("");`
Ctrl + Alt + L (⌥⌘L en macOS)	Reformatea el código (sangría y espacios)
Alt + Intro (⌥↩)	Muestra soluciones rápidas para el error subrayado
Ctrl + / (⌘/)	Comenta o descomenta la línea

> **Consejo**
> Pulsa **Ctrl + Alt + L** antes de entregar cualquier ejercicio. Un código bien sangrado se lee mucho mejor y facilita encontrar llaves mal cerradas.

---

3. Estructura de un programa Java

3.1. Forma clásica

```java
public class Saludo {
    public static void main(String[] args) {
        System.out.println("Hola, mundo");
    }
}
```

Parte	Significado
`public class Saludo`	Define una clase pública llamada `Saludo`. **El archivo debe llamarse `Saludo.java`.**
`{ … }`	Las llaves delimitan el contenido de la clase y del método.
`public static void main(String[] args)`	Punto de entrada: la JVM empieza a ejecutar aquí.
`System.out.println(...)`	Escribe un texto en la consola y salta de línea.
`;`	Fin de la instrucción.

De momento, copia la cabecera de `main` tal cual. Más adelante (métodos en la UP4, clases y objetos en la UP5) entenderás qué significan `static`, `void` y `String[] args`.

3.2. Forma compacta (Java 25 y posteriores)

Desde Java 25 se puede escribir un programa sencillo sin declarar la clase y con un `main` más corto. Además, la clase `IO` simplifica la entrada y salida:

```
void main() {
    String nombre = IO.readln("¿Cómo te llamas? ");
    IO.println("Hola, " + nombre);
}
```

Es útil para los primeros ejercicios, y la web de la UP2 la acepta. Aun así, en clase usaremos sobre todo la forma clásica, porque es la que encontrarás en la mayoría de proyectos y la que necesitaremos cuando trabajemos con objetos.

> **Cuidado**
> La forma compacta solo funciona con un JDK 25 o superior. Si IntelliJ marca error en `void main()`, revisa en *File → Project Structure → Project* que el SDK y el *Language level* sean 25.

3.3. Paquetes y orden de un archivo

Un **paquete** es una carpeta lógica que agrupa clases relacionadas (como `java.util`, donde vive `Scanner`). En un proyecto propio se declara con `package` en la **primera línea** del archivo y el nombre debe coincidir con la carpeta dentro de `src`. Si creas la clase directamente en `src`, queda en el *paquete por defecto* y no hace falta escribir nada. En esta unidad es lo habitual; en la UP6 organizaremos las clases en paquetes propios.

El orden dentro de un archivo es siempre el mismo: primero `package` (si lo hay), luego los `import` y después la clase.

```
package ejercicios.up2;      // el archivo está en src/ejercicios/up2/

import java.util.Scanner;

public class Saludo {
    public static void main(String[] args) {
        System.out.println("Hola");
    }
}
```

Las clases de `java.lang` (`String`, `System`, `Math`, `Integer`…) se importan solas; `Scanner` (`java.util`) no, por eso necesita su `import`.

> **Error típico**
> Llamar al archivo `saludo.java` y a la clase `Saludo`, o poner la instrucción fuera de `main`. Java exige que el nombre del archivo coincida exactamente con el de la clase pública, mayúsculas incluidas.

4. Comentarios, identificadores y convenciones

4.1. Comentarios

```
// Comentario de una línea

/* Comentario
   de varias líneas */

/**
 * Comentario de documentación (Javadoc).
 * Se usa para documentar clases y métodos.
 */
```

El comentario **Javadoc** (`/** … */`) se coloca justo antes de una clase o método y admite etiquetas como `@author` o `@version`. IntelliJ lo muestra al pasar el ratón (Ctrl + Q) y la herramienta `javadoc` genera con ellos la documentación HTML (lo haremos en la UP6). Los demás comentarios los ignora el compilador.

```
/**
 * Calcula el precio de un producto con IVA.
 * @author Tu nombre
 * @version 1.0
 */
public class PrecioConIva {
    public static void main(String[] args) {
        // aquí iría el código
    }
}
```

Comenta el **porqué** y no lo evidente. `// suma 1 a i` no aporta nada; `// el primer día del mes es el 1, no el 0` sí.

4.2. Identificadores

Un identificador es el nombre que das a una variable, constante, método o clase.

- Puede contener letras, dígitos, `_` y `$`, pero **no puede empezar por un dígito**.
- No puede coincidir con una palabra reservada (`int`, `class`, `if`, `public`…).
- Distingue mayúsculas y minúsculas.

Elemento	Convención	Ejemplos
Variables y métodos	*lowerCamelCase*	`precioFinal`, `numeroAlumnos`, `calcularMedia`
Clases	*UpperCamelCase*	`Saludo`, `CalculadoraIva`
Constantes	MAYÚSCULAS con `_`	`IVA`, `PRECIO_MAXIMO`

> **Nombres significativos**
>
> `double p1, p2;` obliga a adivinar. `double precioSinIva, precioConIva;` se explica solo. Evita también tildes y la ñ en los identificadores: Java las admite, pero pueden dar problemas al cambiar de sistema operativo.

---

5. Variables y tipos de datos

Una **variable** es un espacio de memoria con nombre, tipo y valor. En Java se declara indicando primero el tipo:

```
int edad;              // declaración
edad = 18;             // asignación
double altura = 1.75;  // declaración con inicialización
```

5.1. Tipos primitivos

Tipo	Ocupa	Rango / valores	Uso habitual
`byte`	1 byte	−128 a 127	Datos binarios
`short`	2 bytes	−32 768 a 32 767	Poco usado
`int`	4 bytes	≈ ±2 100 millones	**Enteros por defecto**
`long`	8 bytes	≈ ±9·10¹⁸	Enteros muy grandes (`3_000_000_000L`)
`float`	4 bytes	~7 cifras significativas	Poco usado (`2.5f`)
`double`	8 bytes	~15 cifras significativas	**Decimales por defecto**
`char`	2 bytes	Un carácter Unicode	`'A'`, `'7'`, `'ñ'`
`boolean`	—	`true` / `false`	Condiciones

5.2. El tipo `String`

`String` no es un tipo primitivo, sino una **clase** que representa cadenas de texto (por eso empieza por mayúscula). Se escribe con comillas dobles; un `char` va entre comillas simples.

```
String ciclo = "DAW";
char grupo = 'A';
String mensaje = "Estudio " + ciclo + " en el grupo " + grupo;
```

El operador `+` con un `String` **concatena**: une textos y convierte a texto los números que encuentre. Un `String` sabe cuántos caracteres tiene: `ciclo.length()` devuelve `3` y `"".length()` devuelve `0`.

Límites y precisión

Un `int` llega hasta `Integer.MAX_VALUE` (2147483647). Si te pasas, Java **no avisa**: el número «da la vuelta» y `Integer.MAX_VALUE + 1` vale `-2147483648`. Para valores mayores usa `long`. Por otro lado, los `double` son aproximados: `0.1 + 0.2` da `0.30000000000000004`. Por eso se muestran con `printf("%.2f")` y no se comparan con `==`.

5.3. ¿Qué tipo elijo?

Dato	Tipo recomendado
Edad, unidades, número de alumnos	`int`
Precio, nota, temperatura, altura	`double`
Nombre, DNI, teléfono, código postal	`String` (no se opera con ellos)
Respuesta sí/no, aprobado/suspendido	`boolean`
Una letra suelta (opción de menú)	`char`

Error clásico

Un teléfono o un código postal **no** son números aunque solo tengan dígitos: no se suman ni se multiplican, y un `int` perdería el cero inicial de `08001`. Guárdalos como `String`.

5.4. Inferencia con `var`

Dentro de un método puedes escribir `var` y Java deduce el tipo por el valor inicial: `var total = 0.0;` crea un `double`. Úsalo solo cuando el tipo sea evidente; al principio es mejor escribir el tipo explícitamente.

5.5. Ámbito de una variable

El **ámbito** (*scope*) de una variable es la parte del programa donde existe y se puede usar: **desde la línea donde se declara hasta la llave `}` que cierra el bloque** en el que está. Una variable declarada en `main` vive hasta el final de `main`; una declarada dentro de unas llaves interiores desaparece al cerrarlas. Lo verás constantemente con `if` y `for` en la UP3; aquí lo vemos con un bloque simple:

```
public class Ambito {
    public static void main(String[] args) {
        int total = 10;              // visible en todo main, desde esta línea
        {
            int temporal = total * 2; // solo existe dentro de este bloque
            System.out.println(temporal);
        }
        System.out.println(total);
        // System.out.println(temporal);  // ERROR: cannot find symbol
    }
}
```

Salida:

```
20
10
```

Otras dos reglas relacionadas: no se puede declarar dos veces el mismo nombre en el mismo ámbito (`variable edad is already defined in method main(String[])`) y una variable local hay que **darle un valor antes de usarla** (`variable x might not have been initialized`).

---

6. Constantes y literales

6.1. Constantes con `final`

Una constante es una variable cuyo valor no puede cambiar después de asignarlo:

```
final double IVA = 0.21;
final int MAX_INTENTOS = 3;
```

Si intentas `IVA = 0.10;`, el compilador da error. Las constantes evitan los **números mágicos**: un `0.21` repartido por el código obliga a buscarlo en todas partes si cambia el impuesto.

6.2. Literales

Literal	Tipo	Ejemplo
Entero	`int`	`42`, `-7`, `1_000_000`
Entero largo	`long`	`9_000_000_000L`
Decimal	`double`	`3.14`, `2e3` (2000.0)
Decimal simple	`float`	`3.14f`
Carácter	`char`	`'a'`, `'\n'`
Texto	`String`	`"Hola"`
Lógico	`boolean`	`true`, `false`

En el código, el separador decimal **siempre es el punto**: `3.5`, nunca `3,5`.

> **Errores típicos con literales**
>
> `long grande = 3000000000;` falla con `integer number too large`: el literal es un `int` y no cabe; hay que escribir `3000000000L`. `float f = 3.14;` falla con `possible lossy conversion from double to float`: el literal decimal es un `double`; hay que escribir `3.14f`. Y `final int K = 1; K = 2;` da `cannot assign a value to final variable K`.

7. Operadores y expresiones

7.1. Aritméticos

Operador	Significado	Ejemplo	Resultado
`+`	Suma (o concatenación)	`7 + 2`	`9`
`-`	Resta	`7 - 2`	`5`
`*`	Multiplicación	`7 * 2`	`14`
`/`	División	`7 / 2`	`3` (¡entera!)
`/`	División	`7.0 / 2`	`3.5`
`%`	Resto	`7 % 2`	`1`

7.2. La división entera

Si los **dos** operandos son enteros, `/` hace división entera y descarta los decimales. Esto es útil (separar horas y minutos, cifras de un número) y también una fuente de errores.

```
int totalSegundos = 3725;
int horas = totalSegundos / 3600;            // 1
int minutos = totalSegundos % 3600 / 60;     // 2
int segundos = totalSegundos % 60;           // 5

int suma = 7 + 8;
double mediaMal = suma / 2;                  // 7.0  (se divide como int)
double mediaBien = suma / 2.0;               // 7.5
```

> **Recuerda**
>
> `double media = suma / 2;` **no** arregla el problema: primero se calcula `suma / 2` con enteros (7) y después se guarda como `7.0`. Uno de los operandos debe ser decimal antes de dividir.

7.3. Asignación compuesta, incremento y decremento

Expresión	Equivale a
`x += 5`	`x = x + 5`
`x -= 2`	`x = x - 2`
`x *= 3`	`x = x * 3`
`x /= 2`	`x = x / 2`
`x %= 4`	`x = x % 4`
`x++` / `++x`	`x = x + 1`
`x--` / `--x`	`x = x - 1`

`x++` usa el valor y después incrementa; `++x` incrementa y después usa el valor. Para evitar confusiones, escríbelos en una línea propia (`contador++;`).

7.4. Relacionales y lógicos

Devuelven un `boolean` y los usarás constantemente en la UP3.

Operador	Significado	Operador	Significado
`==`	igual	`&&`	Y (ambas ciertas)
`!=`	distinto	`||`	O (al menos una cierta)
`<` `<=`	menor, menor o igual	`!`	NO (niega)
`>` `>=`	mayor, mayor o igual		

```java
int edad = 20;
boolean mayorDeEdad = edad >= 18;                // true
boolean joven = edad >= 18 && edad < 30;         // true
```

> **No confundir**
> `=` asigna y `==` compara números. Para comparar textos **no** se usa `==`, sino `texto1.equals(texto2)`; lo veremos en la UP3.

7.5. Precedencia

De mayor a menor prioridad: paréntesis → `++ -- !` → `* / %` → `+ -` → relacionales → `==` `!=` → `&&` → `||` → asignaciones. Ante la duda, **usa paréntesis**: `(a + b) / 2` no es lo mismo que `a + b / 2`.

7.6. Ejemplos de expresiones evaluadas

Con `int a = 7, b = 2;` estos son los resultados reales:

Expresión	Resultado	Por qué
`a / b`	`3`	Entero entre entero: se descartan los decimales
`a % b`	`1`	7 = 3·2 + 1
`a + b / 2`	`8`	Primero `b / 2` = 1 y después 7 + 1
`(a + b) / 2`	`4`	9 / 2 con enteros
`-7 / 2` y `-7 % 2`	`-3` y `-1`	La división entera trunca hacia cero; el resto toma el signo del dividendo
`7.0 / 0`	`Infinity`	Con decimales no hay error: sale infinito
`7 / 0`	excepción	Con enteros el programa se detiene: `ArithmeticException: / by zero`

Y un ejemplo de incremento: observa en qué momento cambia cada variable.

```
int x = 5;
int y = x++;      // y recibe 5 y después x pasa a 6
int z = ++x;      // x pasa a 7 y después z recibe 7
System.out.println(x + " " + y + " " + z);
```

Salida:

```
7 5 7
```

---

8. Conversiones de tipo

Un dato tiene un tipo y a veces necesitamos verlo con otro: usar un `int` en una cuenta con decimales, quedarnos con la parte entera de un `double`, mostrar un número como texto o convertir en número lo que el usuario ha escrito. Este apartado explica las cuatro situaciones, empezando por la regla general: **si no se pierde información, Java convierte solo; si se puede perder, hay que pedirlo expresamente** con un *casting*.

8.1. Conversión implícita (ampliación)

Java convierte automáticamente de un tipo «pequeño» a otro «grande» porque no se pierde información. El orden es: `byte → short → int → long → float → double` (y `char → int`).

```
int entero = 9;
double d = entero;                // el int se convierte en double
System.out.println(d);
System.out.println(entero / 2);   // int / int
System.out.println(entero / 2.0); // int / double
System.out.println(entero * 1.5); // int * double
```

Salida:

```
9.0
4
4.5
13.5
```

La conversión también ocurre dentro de las expresiones: cuando una operación mezcla un `int` y un `double`, Java convierte el `int` a `double` **antes** de operar y el resultado es `double`. Por eso `9 / 2.0` vale 4.5, mientras que `9 / 2`, con los dos operandos enteros, vale 4. Además, `byte`, `short` y `char` se convierten a `int` al operar con ellos.

8.2. Conversión explícita (*casting*)

Para ir de un tipo grande a uno pequeño hay que escribir el tipo de destino entre paréntesis delante del valor: `(int) valor`. Con ello le dices al compilador «sé que puedo perder información». Sin el casting, el código ni compila (`possible lossy conversion from double to int`).

```java
double real = 2.75;
System.out.println((int) real);            // se trunca, no se redondea
System.out.println((int) -2.75);           // hacia cero
System.out.println(Math.round(real));      // redondeo al más cercano
int redondeado = (int) Math.round(real);   // Math.round devuelve long
```

Salida:

```
2
-2
3
```

Un `(int)` **no redondea: elimina los decimales** (3.99 pasa a 3). Si quieres redondear, usa `Math.round`, que devuelve un `long`, y si lo necesitas en un `int` hay que hacer otro casting, como en la última línea.

A qué afecta el casting

El casting se aplica solo a lo que tiene **justo a su derecha**. Por eso estas dos líneas son distintas:

```java
int a = 7, b = 2;
System.out.println((double) a / b);      // 3.5  (convierte a, luego divide)
System.out.println((double) (a / b));    // 3.0  (divide enteros y luego convierte)
```

Salida:

```
3.5
3.0
```

Asignación compuesta y desbordamiento

Los operadores como `+=` incluyen un casting implícito: `int x = 5; x += 2.7;` compila y deja `x` en `7` (calcula 7.7 y trunca). En cambio `x = x + 2.7;` **no compila**. Y cuando el valor no cabe en el tipo de

destino, el casting no avisa y el resultado es otro número:

```
long grande = 3_000_000_000L;
System.out.println((int) grande);   // no cabe en un int
System.out.println((byte) 130);     // un byte llega solo hasta 127
```

Salida:

```
-1294967296
-126
```

8.3. Resumen: int, double y String

Estas son las conversiones que más usarás en esta unidad. Las tres primeras filas son casting (o conversión automática); las demás usan métodos de clases predefinidas:

De	A	Cómo se escribe	Ejemplo
`int`	`double`	Automática: `double d = n;`	7 → `7.0`
`double`	`int` (trunca)	Casting: `int n = (int) x;`	3.99 → `3`
`double`	`int` (redondea)	`int n = (int) Math.round(x);`	3.99 → `4`
`int` / `double`	`String`	`String.valueOf(x)`, `Integer.toString(n)`, `Double.toString(x)` o `"" + x`	3.99 → `"3.99"`
`double`	`String` con formato	`String.format("%.2f", x)`	3.99 → `"3,99"` (según la región)
`String`	`int`	`Integer.parseInt(s)`	`"18"` → `18`
`String`	`double`	`Double.parseDouble(s)`	`"19.95"` → `19.95`

> **Un String no se convierte con casting**
> `(int) "18"` da `incompatible types: String cannot be converted to int`. El casting solo vale entre tipos numéricos (y `char`). Para pasar de texto a número hay que *interpretar* el texto, y para eso están los métodos `parseInt` y `parseDouble`. Del mismo modo, `int t = "5";` y `String t = 5;` no compilan.

8.4. De número a texto

```java
double precio = 19.5;
String t1 = String.valueOf(precio);
String t2 = Double.toString(precio);
String t3 = "" + precio;
String t4 = Integer.toString(42);
String t5 = String.format("%.2f", precio);
System.out.println(t1 + " " + t2 + " " + t3 + " " + t4 + " " + t5);
System.out.println(t1.length() + " " + t5.length());
System.out.println("3" + 4);
System.out.println(3 + 4);
```

Salida (en un ordenador configurado en español):

```
19.5 19.5 19.5 42 19,50
4 5
34
7
```

Las tres primeras formas dan el mismo texto, `"19.5"`, siempre con punto. `String.format` funciona como `printf` pero **devuelve** el texto en lugar de mostrarlo, y usa la coma decimal en un equipo en español (por eso `t5` tiene 5 caracteres). Una vez convertido a `String` puedes preguntar su longitud con `length()`. Fíjate también en que `"3" + 4` es el texto `"34"`, mientras que `3 + 4` es el número 7.

8.5. De texto a número

```java
int edad = Integer.parseInt("18");
double precio = Double.parseDouble("19.95");
System.out.println(edad + 1);
System.out.println(precio + 1);
System.out.println(Integer.parseInt("18") + Integer.parseInt("4"));
System.out.println("18" + "4");
```

Salida:

```
19
20.95
22
184
```

Las dos últimas líneas muestran la diferencia: sumar números (22) o *pegar textos* (184). Convertir a número es imprescindible antes de operar con un dato leído como `String` (por ejemplo, con `IO.readln` o `nextLine()`).

Si el texto no es un número válido, `parseInt` y `parseDouble` lanzan una excepción (`NumberFormatException`) y el programa se detiene. Son errores típicos:

Texto	Resultado de `Integer.parseInt`
`"hola"`	Error: `For input string: "hola"`
`"3.5"`	Error: un `int` no admite decimales (usa `parseDouble`)
`" 18"` (con espacio)	Error. Se arregla con `Integer.parseInt(texto.trim())`, que quita los espacios de los extremos
`""` (vacío)	Error
`"19,95"` con `parseDouble`	Error: `parseDouble` espera **punto** decimal siempre

Cómo evitar el fallo comprobando antes el texto lo aprenderás en la UP3, cuando existan condicionales y bucles.

8.6. Caracteres y números

Un `char` guarda internamente un número (su código Unicode), así que se convierte fácilmente a `int` y al revés. No confundas el carácter `'7'` con el número 7:

```
char c = 'A';
int codigo = c;                      // conversión automática
System.out.println(codigo);
System.out.println(c + 1);           // char + int = int
System.out.println((char) (c + 1));  // casting de vuelta a char
char siete = '7';
System.out.println(siete - '0');     // de carácter-dígito a número
```

Salida:

```
65
66
B
7
```

La resta `'7' - '0'` funciona porque los dígitos están ordenados en la tabla de códigos (`'0'` es 48 y `'7'` es 55).

---

9. Salida por pantalla

9.1. `print` y `println`

```
System.out.print("Sin salto ");
System.out.println("con salto al final");
System.out.println("Total: " + 25 + 10);     // Total: 2510
System.out.println("Total: " + (25 + 10));   // Total: 35
```

Observa las dos últimas líneas: el `+` se evalúa de izquierda a derecha. En cuanto aparece un `String`, el resto se concatena como texto. Los paréntesis fuerzan la suma.

9.2. Secuencias de escape

Secuencia	Resultado
`\n`	Salto de línea
`\t`	Tabulación
`\"`	Comillas dobles dentro de un texto
`\\`	Barra invertida

9.3. Salida con formato: `printf`

```
String producto = "Teclado";
int unidades = 2;
double precio = 25.5;
System.out.printf("%-10s %3d uds. %8.2f €%n", producto, unidades, unidades * precio);
```

Especificador	Significado
`%d`	Entero
`%f`, `%.2f`	Decimal, con 2 decimales
`%s`	Texto
`%c`	Carácter
`%n`	Salto de línea
`%8.2f`	Ancho mínimo 8, 2 decimales (alineado a la derecha)
`%-10s`	Ancho mínimo 10 alineado a la izquierda

Con los datos del ejemplo anterior la salida es (en un equipo en español):

```
Teclado      2 uds.    51,00 €
```

Cada especificador consume un argumento **en orden** y debe corresponder a su tipo. También hay relleno con ceros: `%05d` con 42 da `00042`, y `%5d` da `42`.

> **Decimales con coma**
> `printf` usa la configuración regional del ordenador. En un equipo configurado en español verás `51,00` en lugar de `51.00`. Es correcto: es solo la forma de mostrarlo.

9.4. `String.format` y errores de formato

`String.format` usa exactamente los mismos especificadores que `printf`, pero en vez de escribir en pantalla **devuelve un** `String` que puedes guardar o reutilizar:

```
String linea = String.format("%s: %.2f €", "Total", 121.0);
System.out.println(linea);
System.out.printf("%s mide %.1f m y tiene %d años%n", "Ana", 1.7, 20);
System.out.println("Con concatenación: " + 1.75);
```

Salida (equipo en español):

```
Total: 121,00 €
Ana mide 1,7 m y tiene 20 años
Con concatenación: 1.75
```

Observa la diferencia: `printf` y `String.format` usan la coma decimal de la región, mientras que concatenar con `+` muestra **siempre** el punto.

> **Error típico**
> Usar un especificador que no corresponde al tipo. `System.out.printf("%d", 2.5)` falla al ejecutar con `IllegalFormatConversionException: d != java.lang.Double` y `printf("%.2f", 3)` con `f != java.lang.Integer`. Usa `%d` para enteros y `%f` para decimales. Otro error frecuente es olvidar `%n` (o `println`) y que todo salga en la misma línea.

---

10. Entrada por teclado

10.1. La clase `Scanner`

```
import java.util.Scanner;

public class Datos {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Nombre: ");
        String nombre = teclado.nextLine();
        System.out.print("Edad: ");
        int edad = teclado.nextInt();
        System.out.print("Altura en metros: ");
        double altura = teclado.nextDouble();

        System.out.println(nombre + " tiene " + edad + " años y mide " + altura + " m");
    }
}
```

Método	Lee
`nextLine()`	Una línea completa de texto (con espacios)
`next()`	Una palabra (hasta el primer espacio)
`nextInt()`	Un `int`
`nextDouble()`	Un `double`
`nextBoolean()`	`true` / `false`

La línea `import java.util.Scanner;` va al principio del archivo, antes de la clase. IntelliJ la añade sola si pulsas Alt + Intro sobre `Scanner`.

Ejemplo de ejecución del programa anterior (lo que escribe el usuario va en la entrada; la altura se teclea con coma porque el equipo está en español):

```
Nombre: Ana Pérez
Edad: 20
Altura en metros: 1,75
Ana Pérez tiene 20 años y mide 1.75 m
```

Fíjate en que al mostrar la altura con `+` aparece el punto (`1.75`), aunque se tecleó con coma: la lectura depende de la región, la concatenación no. También puedes teclear varios números en una línea separados por espacios (`3 4`) cuando haces dos llamadas seguidas a `nextInt()`.

10.2. Trampa 1: `nextInt()` seguido de `nextLine()`

`nextInt()` lee el número, pero deja en el teclado el **Intro** que pulsaste. Si después llamas a `nextLine()`, este lee esa línea vacía y parece que «se salta» la pregunta.

```java
System.out.print("Edad: ");
int edad = teclado.nextInt();
teclado.nextLine();                  // consume el Intro pendiente
System.out.print("Nombre completo: ");
String nombre = teclado.nextLine();
```

Otra opción es leer siempre líneas y convertir: `int edad = Integer.parseInt(teclado.nextLine());`.

10.3. Trampa 2: punto o coma decimal

`nextDouble()` también depende de la configuración regional. En un ordenador en español espera **coma** (`1,75`) y falla con `1.75` (`InputMismatchException`). Si quieres que siempre acepte el punto:

```java
Scanner teclado = new Scanner(System.in).useLocale(java.util.Locale.US);
```

10.4. Entrada con `IO` (Java 25)

En la forma compacta, `IO.readln("Mensaje: ")` muestra el mensaje y devuelve la línea escrita como `String`. Para números, conviértela:

```java
int edad = Integer.parseInt(IO.readln("Edad: "));
```

---

11. La clase `Math`

`Math` agrupa funciones matemáticas listas para usar. No hace falta `import`.

Son **métodos estáticos**: pertenecen a la clase y se llaman escribiendo `Clase.método(...)`, **sin crear ningún objeto** con `new`. Ya has usado más: `Integer.parseInt`, `Double.parseDouble` y `String.valueOf`. Compáralo con `teclado.nextInt()`, que no es estático: necesita el objeto `teclado` que creamos con `new Scanner(...)`. Además, `Math.PI` no es un método sino una **constante estática** (sin paréntesis). Cómo se crean tus propios métodos estáticos lo verás en la UP4.

Expresión	Resultado	Para qué
`Math.PI`	3.141592...	Constante π
`Math.pow(2, 10)`	1024.0	Potencia
`Math.sqrt(16)`	4.0	Raíz cuadrada
`Math.abs(-7)`	7	Valor absoluto
`Math.max(3, 9)` / `Math.min(3, 9)`	9 / 3	Mayor y menor
`Math.round(2.5)`	3	Redondeo al entero más cercano
`Math.floor(2.9)` / `Math.ceil(2.1)`	2.0 / 3.0	Redondeo hacia abajo / arriba
`Math.random()`	[0, 1)	Número aleatorio decimal

Para redondear a dos decimales: `Math.round(valor * 100) / 100.0`. Por ejemplo, con `3.14159`: `valor * 100` es 314.159, `Math.round` da 314 y `314 / 100.0` da `3.14` (el `100.0` evita la división entera).

Detalles que conviene conocer: `Math.round(2.5)` es 3 pero `Math.round(-2.5)` es `-2` (redondea hacia arriba en los empates); `Math.floor(-2.5)` es `-3.0` y `Math.ceil(-2.5)` es `-2.0`; `Math.max(3, 9.5)` devuelve `9.5` (un `double`) y `Math.sqrt(2)` es `1.4142135623730951`.

> **Error típico**
>
> Guardar el resultado de `Math` en un `int` sin casting. `int r = Math.round(2.5);` no compila (`possible lossy conversion from long to int`, porque `round` con un `double` devuelve `long`) y `int p = Math.pow(2, 3);` tampoco (`pow` siempre devuelve `double`). Soluciones: `int r = (int) Math.round(2.5);` y `int p = (int) Math.pow(2, 3);` (vale 8).

Ejemplo: área y perímetro de un círculo

Entrada: el radio. **Proceso:** área = π·r², perímetro = 2·π·r. **Salida:** ambos valores con 2 decimales.

```java
import java.util.Scanner;

public class Circulo {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.print("Radio: ");
        double radio = teclado.nextDouble();

        double area = Math.PI * Math.pow(radio, 2);
        double perimetro = 2 * Math.PI * radio;

        System.out.printf("Área: %.2f%n", area);
        System.out.printf("Perímetro: %.2f%n", perimetro);
    }
}
```

Prueba: con radio 2 → área 12,57 y perímetro 12,57.

---

12. Errores frecuentes y cómo leerlos

Hay tres tipos de errores, y cada uno se detecta en un momento distinto.

Tipo	Cuándo aparece	Ejemplo	Cómo se resuelve
De compilación	Antes de ejecutar; IntelliJ lo subraya en rojo	Falta `;`, variable no declarada, tipos incompatibles	Lee el mensaje y la línea indicada.
De ejecución	Al ejecutar; el programa se detiene con una excepción	Escribir `hola` cuando se espera un número	Lee la primera línea de la traza (`Exception in thread "main" …`).
Lógico	El programa termina, pero el resultado es incorrecto	Media con división entera	Prueba con datos cuyo resultado conoces; haz una traza.

Mensaje	Causa probable
`';' expected`	Falta el punto y coma al final de la línea anterior.
`cannot find symbol`	Nombre mal escrito (mayúsculas) o variable no declarada.
`incompatible types: possible lossy conversion from double to int`	Guardas un decimal en un `int` sin *casting*.
`variable x might not have been initialized`	Usas una variable antes de darle valor.
`class Saludo is public, should be declared in a file named Saludo.java`	El nombre de la clase y el del archivo no coinciden.
`InputMismatchException`	El dato tecleado no es del tipo esperado (o punto/coma decimal).
`NumberFormatException`	`parseInt` / `parseDouble` con un texto que no es un número.
`integer number too large`	Literal entero mayor que un `int`: añade `L` (`3000000000L`) y usa `long`.
`incompatible types: String cannot be converted to int` (o `int cannot be converted to String`)	Mezclas texto y número en una asignación. Convierte con `parseInt` o `String.valueOf`; un *casting* no sirve.
`variable edad is already defined in method main(String[])`	Declaras dos veces el mismo nombre en el mismo ámbito.
`cannot assign a value to final variable IVA`	Intentas cambiar una constante.
`ArithmeticException: / by zero`	División o resto entre enteros con divisor 0.
`IllegalFormatConversionException`	Especificador de `printf` que no corresponde al tipo del argumento (`%d` con un decimal).

13. Ejemplos resueltos paso a paso

13.1. Precio con IVA

Problema: pedir el precio de un producto y mostrar el IVA (21 %) y el total.

Idea: constante para el IVA; `iva = precio × IVA`; `total = precio + iva`.

```java
import java.util.Scanner;

public class PrecioConIva {
    public static void main(String[] args) {
        final double IVA = 0.21;
        Scanner teclado = new Scanner(System.in);

        System.out.print("Precio sin IVA: ");
        double precio = teclado.nextDouble();

        double importeIva = precio * IVA;
        double total = precio + importeIva;

        System.out.printf("IVA (21 %%): %.2f €%n", importeIva);
        System.out.printf("Total: %.2f €%n", total);
    }
}
```

Prueba: 100 → IVA 21,00 € y total 121,00 €. Observa `%%`: así se escribe un símbolo `%` dentro de `printf`.

13.2. Intercambio de dos variables

Problema: intercambiar los valores de `a` y `b`.

Idea: si hacemos `a = b` perdemos el valor de `a`. Necesitamos una variable auxiliar.

```java
public class Intercambio {
    public static void main(String[] args) {
        int a = 8;
        int b = 3;
        int aux = a;   // guardamos a
        a = b;         // a vale 3
        b = aux;       // b vale 8
        System.out.println("a = " + a + ", b = " + b);
    }
}
```

Instrucción	a	b	aux
inicio	8	3	—
`aux = a`	8	3	8
`a = b`	3	3	8
`b = aux`	3	8	8

13.3. Segundos a horas, minutos y segundos

```java
import java.util.Scanner;

public class Tiempo {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.print("Segundos: ");
        int total = teclado.nextInt();

        int horas = total / 3600;
        int resto = total % 3600;
        int minutos = resto / 60;
        int segundos = resto % 60;

        System.out.println(horas + " h " + minutos + " min " + segundos + " s");
    }
}
```

Prueba: 3725 → `1 h 2 min 5 s`.

13.4. Conversiones de tipo paso a paso

Problema: leer un entero y un número real, y mostrar el entero como `double`, el real truncado y redondeado a entero, y el real como texto con su longitud.

Idea: `int → double` es automático; `double → int` necesita casting (y `Math.round` si queremos redondear); `double → String` con `String.valueOf`. Para que el programa acepte el punto decimal en cualquier ordenador usamos `useLocale(Locale.US)`.

```java
import java.util.Locale;
import java.util.Scanner;

public class Conversiones {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in).useLocale(Locale.US);

        System.out.print("Entero: ");
        int entero = teclado.nextInt();
        System.out.print("Real: ");
        double real = teclado.nextDouble();

        double enteroComoReal = entero;                // automática
        int realTruncado = (int) real;                 // casting: trunca
        int realRedondeado = (int) Math.round(real);   // redondea y luego casting
        String realTexto = String.valueOf(real);       // double -> String

        System.out.println("Entero como double: " + enteroComoReal);
        System.out.println("Real truncado: " + realTruncado);
        System.out.println("Real redondeado: " + realRedondeado);
        System.out.println("Real como texto: \"" + realTexto + "\" (" + realTexto.length() + "
caracteres)");
        System.out.println("Media sin casting: " + (entero + realTruncado) / 2);
        System.out.println("Media con casting: " + (entero + realTruncado) / 2.0);
    }
}
```

Prueba: con 9 y 2.75:

```
Entero como double: 9.0
Real truncado: 2
Real redondeado: 3
Real como texto: "2.75" (4 caracteres)
Media sin casting: 5
Media con casting: 5.5
```

Las dos últimas líneas recuerdan por qué importa el tipo: (9 + 2) / 2 con enteros da 5, y dividir entre `2.0` da 5.5.

13.5. Media de tres notas

Problema: calcular la media de tres notas enteras con decimales.

Idea: sumar y dividir entre 3, pero cuidado con la división entera.

```java
public class Media {
    public static void main(String[] args) {
        int n1 = 7, n2 = 8, n3 = 8;
        double mal = (n1 + n2 + n3) / 3;        // 23 / 3 con enteros
        double bien = (n1 + n2 + n3) / 3.0;     // 23 / 3.0
        System.out.println(mal);
        System.out.println(bien);
        System.out.printf("%.2f%n", bien);
    }
}
```

Prueba (equipo en español):

```
7.0
7.666666666666667
7,67
```

La primera versión pierde los decimales aunque el resultado se guarde en un `double`: la división ya se había hecho con enteros. La segunda divide entre `3.0`, y `%.2f` redondea solo al *mostrar*.

---

14. Banco de ejercicios

Los ejercicios de esta unidad están en la web de la UP2, con pistas en la plantilla, validación por criterios y justificante:

- **P2.1 · Java paso a paso (12 ejercicios):** nombre y edad, suma, cinco operaciones, área del rectángulo, círculo, intercambio, informe de ventas, IVA fijo, IVA variable, euros a pesetas, las dos piscinas y un ejercicio de conversiones de tipo (*casting*: `int → double`, `double → int` y `double → String`; repasa los apartados 8.1 a 8.4).
- **P2.2 · Ampliación de sentencias simples (12 retos):** ticket con propina, segundos a h/min/s, cajero, notas ponderadas, temperaturas, cifras de un número, viaje, carrera, rotación de tres variables, factura de la luz y distancia entre puntos, entre otros.

> **Método de trabajo**
>
> Resuelve en IntelliJ, ejecuta con varios datos (incluido algún caso límite), pega el código en la web, valida y corrige. La web no compila tu programa: la prueba de verdad es ejecutarlo.

15. Referencia rápida

```java
import java.util.Scanner;                    // al principio del archivo

public class Plantilla {
    public static void main(String[] args) {
        final double CONSTANTE = 0.21;       // constante
        Scanner teclado = new Scanner(System.in);

        int entero = teclado.nextInt();      // leer entero
        double decimal = teclado.nextDouble(); // leer decimal
        teclado.nextLine();                  // limpiar el Intro
        String texto = teclado.nextLine();   // leer línea

        double resultado = entero * decimal * CONSTANTE;
        System.out.println("Texto: " + texto);
        System.out.printf("Resultado: %.2f%n", resultado);
    }
}
```

Necesito…	**Escribo…**
Resto de una división	`a % b`
División con decimales entre enteros	`(double) a / b` o `a / 2.0`
Potencia	`Math.pow(base, exp)`
Raíz cuadrada	`Math.sqrt(x)`
Redondear a 2 decimales al mostrar	`printf("%.2f", x)`
Convertir texto a número	`Integer.parseInt(s)`, `Double.parseDouble(s)`
Truncar un decimal	`(int) x`
Pasar un `int` a `double`	`double d = n;` (automático)
Número a texto	`String.valueOf(x)` o `"" + x`
Texto con 2 decimales como `String`	`String.format("%.2f", x)`
Redondear un decimal a entero	`(int) Math.round(x)`
Número de caracteres de un texto	`texto.length()`
Carácter dígito a número	`c - '0'`